

ASSIGNMENT 3

Software Development and Integration

Taxonomy Evaluator: a reproducible scientific tool

Prepared for the scientific software development assignment

GitHub author: thgank

6 October 2026

1. Purpose, scope, and deliverables

Scientific software should make a research procedure inspectable and repeatable. In domain-taxonomy research, a visually plausible hierarchy is insufficient evidence that the generated parent–child relations are correct. An independent evaluation procedure is needed to distinguish structural consistency from agreement with expert knowledge.

This assignment develops *Taxonomy Evaluator*, a separate command-line project inspired by the author’s dissertation system, Taxonomy Builder. The original system creates taxonomies from documents; the new project evaluates exported taxonomies against an external reference. It implements directed-edge precision, recall, and F1, graph diagnostics, reproducible input identification, automated tests, and continuous integration with delivery of a validated artifact.

The work has two parts. This ten-page analytical report discusses development organization, technology selection, version control, CI/CD, and scientific computing practices. The practical repository contains original Python code, synthetic examples, a README, an MIT license, issue templates, tests, a GitHub Actions workflow, and this report’s editable source. The assignment document itself is retained locally rather than republished.

Assignment requirement	Evidence in the submission
Scientific module and documentation	Evaluator, input conventions, runnable examples
Git and public GitHub hosting	Focused commits; public repository and issues
Automated testing and CI/CD	Regression tests; test matrix; delivered ZIP
Technology justification and analysis	Sections 2–8 of this report
Results, limitations, and conclusion	Sections 9–10; demonstration JSON reports

Repository: <https://github.com/thgank/taxonomy-evaluator>

CI/CD: `.github/workflows/ci.yml` in the public repository.

Execution evidence: <https://github.com/thgank/taxonomy-evaluator/actions>

2. Analysis of the dissertation project

The reviewed project is thgank/taxonomy-builder, at revision adc387a5. The repository describes automatic and semi-automatic taxonomy construction from document collections. Its architecture combines a Java/Spring Boot API, Python workers, PostgreSQL, RabbitMQ, and a web application. The documented pipeline includes document import, NLP, term extraction, hierarchy construction, and evaluation. Statistical term ranking, lexical patterns, embeddings, and human review serve different stages. [1]

Inspection of the implementation adds an important qualification: evaluation already exists in the original project. The worker’s evaluation module reports coverage, depth, branching, connectivity, confidence, and review-related statistics. Therefore, recreating an internal graph-quality service would provide limited value. These diagnostics describe properties of a generated hierarchy; they do not by themselves establish exact agreement with an independently curated gold-standard edge set.

The API’s export service provides a useful integration boundary. Its JSON export contains nodes with IDs and labels, plus edges whose parent and child values are IDs. Its CSV export includes parent and child labels, relation, and score. A JSON request with the option to include orphan concepts retains nodes absent from edges. The new evaluator reads both actual export layouts, resolves IDs to labels, and needs no database connection or running worker. [1]

The original repository also contains automated Java, Python, web, and integration checks, together with a quality-regression script. This supports a separation of responsibilities: maintain construction and application-level checks there, while placing reference-based benchmarking in the new project. No source files were copied from the original system.

	Taxonomy Builder	Taxonomy Evaluator
Research role	Generate and review hierarchies	Measure agreement with a reference
Input/output	Documents to versioned taxonomy	Exports and gold edges to JSON report
Runtime	Multiple services and infrastructure	Offline Python command
Evaluation focus	Internal quality and operational checks	Directed-edge errors and independent audit

This is a bounded source review, not a performance or scientific-validity audit of the dissertation system. The latter would require its actual corpora, annotation protocol, experiment configurations, and expert results.

3. Stages and organization of software development

The development process starts with the research question: how closely do generated taxonomic relations match a reference hierarchy? Requirements follow from this question. The tool must read available exports, preserve relation direction, compare unique edges, expose disagreement examples, detect directed cycles, and record the exact evaluated inputs. Nonfunctional requirements are portability, offline execution, clear failure behavior, and a small maintenance burden.

The next stage is design. A file-based integration boundary avoids coupling evaluation to database entities or message queues. The design separates loading, label normalization, reference comparison, structural analysis, and command-line reporting through ordinary functions in one module. This is enough separation to test behavior without introducing a framework or service layer.

Implementation then proceeds in small increments. First, representative export schemas are inspected. Next, the loader and metric calculations are implemented together with hand-checkable example data. Graph checks and safe output behavior follow. Finally, automated tests, CI/CD configuration, and documentation turn the script into a reviewable research artifact. The Git history records these concerns in focused commits rather than presenting one undifferentiated upload.

Verification is a separate activity from implementation. For this project, known edge counts test numerical correctness, reversed relations test directionality, and a graph with two paths to one node tests the definition of depth. Invalid scores, unknown IDs, ambiguous labels, empty inputs, and cyclic hierarchies test failure behavior. Each requirement has an observable outcome rather than relying on a successful demonstration alone.

Delivery consists of publishing the source and producing a runnable, tested ZIP through the workflow. Maintenance uses issues to define defects or research extensions, short feature branches to isolate changes, and pull requests to review effects on metrics and input conventions. An issue should state the expected result and a reproducible example; a research issue should specify the scientific question and acceptance criterion.

A lightweight iterative process suits this assignment. A large sequential specification would be expensive because the reference protocol may evolve. Uncontrolled experimentation would also be risky because changed conventions can silently alter published scores. Iteration is therefore bounded by explicit input schemas, documented metric definitions, automated checks, and versioned releases or commits.

In a larger team, the researcher owns annotation and interpretation, the developer owns implementation, and a reviewer challenges both. In this single-author project those responsibilities remain distinct activities. Before reporting a result, the author should review the gold standard independently of the generated taxonomy to reduce confirmation bias.

4. Criteria and justification for technology selection

Technology selection should respond to actual constraints. Here the inputs are modest JSON/CSV taxonomy exports, the computation is discrete graph and set analysis, and the primary users are researchers already working with Python. There is no requirement for a browser interface, concurrent API traffic, GPU inference, or distributed persistence.

Criterion	Decision and justification
Scientific correctness	Explicit directed-edge definitions and independently known test outcomes
Interoperability	Standard JSON/CSV and the dissertation project’s actual export schemas
Reproducibility	Offline execution; recorded hashes, parameters, and versions
Installation effort	Python 3.11+; zero third-party runtime packages
Maintainability	One module, one test file, ordinary standard-library functions
Performance needs	In-memory sets and adjacency structures for modest exports
Collaboration and delivery	GitHub repository, issues, review workflow, Actions artifacts

Python is chosen because it matches the existing research workflow and supplies the required facilities directly. The `csv` and `json` modules load portable files; `argparse` validates command-line arguments; `hashlib` identifies input bytes; `unicodedata` normalizes labels; `graphlib` supplies topological ordering and cycle detection; `unittest` runs regression checks. `Graphlib`’s ordering permits a longest-path depth calculation on an acyclic graph without implementing a custom topological-sort algorithm. [4,5]

`NetworkX` would offer many additional graph algorithms, but those capabilities are unnecessary for the requested metrics. `pandas` would make tabular manipulation convenient, but the edge-list inputs do not require a dataframe. `NumPy` would not improve the clarity of set intersection. Avoiding these dependencies reduces installation work and removes an unnecessary source of environment variation.

`Java/Spring Boot` remains appropriate to the original service application, but using it for an offline evaluator would introduce compilation and framework setup without a user requirement. A notebook could support exploration, yet an explicit command is easier to run identically in a local terminal and CI. A notebook may later demonstrate findings while calling this same module.

These choices are scoped, not universal recommendations. If future experiments need millions of relations, sophisticated graph measures, interactive annotation, or uncertainty estimation, measured requirements may justify a graph library, streaming storage, or a richer interface. The current tool first establishes an auditable baseline.

5. Version control systems and the GitHub platform

Version control records changes and provides a recoverable history. Git uses a distributed model: a clone contains repository history, and local commits do not require a connection to a central server. This supports research work on a laptop, isolated experiment branches, and later collaboration. Commit messages should explain the scientific behavior that changed, not just name edited files. [2]

System	Model and strengths	Tradeoff for this project
Git	Distributed history; local commits; branching and merging	Selected; fits the existing repository and GitHub workflow
Mercurial	Distributed history and local development	Technically suitable; changing tools gives no demonstrated benefit
Subversion	Centralized revision history and repository access	Useful where centralized control is required; less suitable for offline commits

Mercurial is another distributed system, whereas Subversion organizes collaboration around a central repository. Neither is inherently unscientific. Selection depends on institutional practice, contributor familiarity, and the required collaboration model. Git is the simplest choice here because the author's original project and the requested hosting platform already use it. [8,9]

GitHub is a hosting and collaboration platform, not a competing version control algorithm. Git supplies history, commits, branches, and merges; GitHub supplies a public repository interface, issues, pull requests, workflow execution, and artifact sharing. Local Git history remains useful even if a hosting platform changes. This distinction avoids comparing Git and GitHub as interchangeable systems.

The practical repository uses focused commits for the scientific implementation, CI/CD, and report. The README defines how to reproduce an evaluation, the license states reuse permissions, and issue templates request the information needed to reproduce bugs or define research tasks. The public link makes the submission inspectable by an instructor or scientific collaborator.

A suitable daily process is to create a short branch, change one behavior, run the test command, commit with a descriptive message, and open a pull request. Review should check not only style but also relation orientation, score conventions, and changes to expected results. A changed gold standard is a methodological change and deserves the same review as changed code.

Git should track source, configuration, small licensed examples, and documentation. Credentials, generated caches, private document corpora, and confidential annotations must stay out of public history. Input hashes identify an experiment's files but do not replace data availability, backups, or a documented access protocol.

6. Continuous integration and continuous delivery

Continuous integration checks proposed changes automatically so that errors are discovered before later work depends on them. Continuous delivery prepares validated artifacts that can be used or released. Continuous deployment additionally publishes changes to a running environment automatically. This project needs delivery of a research tool; it has no application server requiring deployment.

GitHub Actions represents automation as a versioned workflow containing event triggers, jobs, and steps. Jobs run on runners and can depend on other jobs. A matrix repeats a job for selected environment combinations. These concepts support a small pipeline that a reviewer can inspect alongside the code. [3]

The workflow is stored in `.github/workflows/ci.yml`. Pushes to main, pull requests, and manual dispatch trigger it. The test job runs the same unittest command used locally on Python 3.11, 3.13, and 3.14. No runtime package installation is needed, so the pipeline checks the supported interpreter range with minimal setup.

The delivery job depends on the full test matrix. It runs the synthetic benchmark with threshold 0.5 and minimum F1 of 0.9. If the gate fails, delivery stops. Otherwise, Python's zipfile module bundles the script, tests, README, license, examples, and evaluation JSON. The artifact upload step publishes the ZIP and JSON under a name containing the Git commit SHA, with 30-day retention. A user can download this validated bundle from a successful run.

This ordering matters. Uploading a package before tests complete could make an unverified tool appear ready for use. The dependency between jobs makes validation a delivery prerequisite. The artifact is a research utility, and the demonstration gate checks software behavior; it is not a quality claim about dissertation-generated taxonomies.

The workflow token has read-only repository permissions. Third-party action references are pinned to inspected immutable revision hashes. User data and credentials are unnecessary for the example run. These choices reduce the workflow's ability to modify the repository or depend on a silently moved action tag. [10]

Jenkins offers broad customization but would require maintaining a server for this small repository. GitLab CI is suitable when the project already lives on GitLab. GitHub Actions is justified by the required GitHub publication and the lack of additional infrastructure. If a future project requires controlled hardware or restricted data, a different runner arrangement may become necessary.

7. Requirements and practices for scientific software

Scientific correctness depends on the research protocol as well as the code. A deterministic implementation can reproducibly calculate a misleading score if its reference taxonomy is incomplete or biased. The protocol must specify the domain, annotation instructions, relation direction, label identity, treatment of multiple inheritance, and resolution of expert disagreement before interpreting results.

For this tool, an edge is ordered from a broader parent concept to a narrower child concept. Exact matching preserves that order: reversing a correct relation produces an error. Labels undergo Unicode NFC normalization, case folding, and whitespace normalization. Synonyms and translated equivalents remain different labels unless the researcher aligns them beforehand. Homonyms need qualified labels, since merging identical text may merge different concepts.

The gold standard must be independent of the evaluated output. Annotators should work from an explicit corpus and domain definition, retain uncertain decisions, and document adjudication. If a threshold is selected, it should be tuned on development data and then frozen before a held-out evaluation. Trying many thresholds on test data and reporting the best result would give an optimistic estimate.

Reproducibility requires retaining the original exports and reference annotations, identifying the software revision, and recording parameters. The evaluator records input-byte SHA-256 hashes, its version, the Python version, normalization rules, and the chosen threshold. It omits timestamps so the same inputs in the same environment produce repeatable reports. A hash confirms file identity; it does not establish provenance or make missing data recoverable.

Wilson and colleagues recommend practical habits such as organized project files, explicit dependencies, reusable scripts, version control, and clear collaboration information. The present repository applies these habits through small examples, documented commands, a license, testable behavior, and an editable report. [7]

Scientific environments should separate tool execution from document production. The evaluator requires only Python's standard library and no external service. The PDF is built with Tectonic and locally installed Times New Roman fonts. Researchers can run the evaluator without installing a typesetting system. Record the environment used for a published result even when no third-party packages are involved.

The synthetic energy-domain examples are deliberately small enough to inspect by hand. They demonstrate arithmetic and software behavior only. A dissertation evaluation still needs real corpora, independently curated references, multiple baselines, and analysis of annotation coverage. Precision and recall measure agreement with the available reference, while structural diagnostics expose different failure modes; neither replaces expert interpretation.

8. Practical module: input, algorithms, and reporting

The module accepts a predicted taxonomy and a gold taxonomy as JSON or CSV. Label-based inputs specify parent and child strings. Taxonomy Builder’s JSON form instead declares node IDs and labels, then refers to IDs in edges. The loader resolves these IDs and rejects missing references. Its CSV form already exposes labels. Only is-a relations are supported; other relation types require a different evaluation definition.

Duplicate edges are evaluated once, retaining the maximum score. Duplicate IDs or normalized node labels in ID-based JSON are rejected as ambiguous. Scores must be finite numbers between zero and one. Missing scores default to one, and the report records unscored row counts. The threshold applies only to predictions and includes scores equal to the threshold; the gold reference remains unchanged.

Let P be the selected predicted edge set and G the gold edge set. The evaluator computes:

$$TP = |P \cap G|, \quad FP = |P \setminus G|, \quad FN = |G \setminus P|.$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2TP}{2TP + FP + FN}.$$

Precision is undefined for empty predictions, recall is undefined for an empty reference, and F1 is conservatively undefined when either edge set is empty. JSON represents undefined values as null. Nonempty disjoint edge sets have F1 zero. Any requested minimum-F1 gate fails when F1 is undefined.

Graph analysis reports roots, leaves, isolated nodes, multiple-parent nodes, weakly connected components, and the largest-component ratio. All declared or observed nodes remain present after threshold filtering. An isolated node counts as both a root and a leaf. Multiple inheritance is allowed. Topological ordering detects directed cycles; for a DAG, dynamic programming calculates the longest root-to-node path in edges, with root depth zero. A cyclic or empty graph has no reported maximum depth.

The JSON output includes both graph analyses and exact false-positive and false-negative edges. Exit code zero denotes passing checks, one denotes a cycle or failed quality gate, and two denotes an invalid input or I/O error. Output replacement is atomic, and an output path cannot overwrite an input. These behaviors support shell-based experiments without hiding invalid evaluations.

Set comparison and graph traversal scale linearly in ordinary cases, apart from deterministic sorting. The full inputs remain in memory, which is appropriate for this module’s intended export sizes. Large-scale streaming is outside the current scope.

9. Validation, demonstration results, and limitations

Validation uses independently known outcomes rather than comparing a function with a duplicate implementation. The synthetic gold reference contains six directed relations. The prediction contains five of those relations, one incorrect shortcut from energy to battery, and no relation from energy storage to thermal storage. This makes the expected true-positive, false-positive, and false-negative counts directly inspectable.

Setting	TP	FP	FN	P	R	F1
All edges	5	1	1	.8333	.8333	.8333
Threshold .5	5	0	1	1.0000	.8333	.9091

Filtering removes the incorrect score-0.40 shortcut, increasing precision without recovering the missing reference relation. The predicted graph at threshold 0.5 has six observed nodes, five edges, one root, one weak component, and depth two. The reference has seven nodes and six edges. These results are saved in the baseline and filtered JSON files under output/results.

The five regression test methods cover known counts and thresholding; JSON/CSV export formats, duplicates, Unicode and isolates; direction, cycles, components and longest depth; empty and invalid data; and command-line gates, invalid arguments, and input protection. The depth test includes a 1,500-edge chain to exercise iterative processing beyond typical recursive traversal limits. The tests pass locally on Python 3.13.3.

The hosted workflow checks three Python versions before delivering the benchmark report and runnable ZIP. Its final execution evidence is available through the repository’s Actions history, where each run identifies the exact commit, individual jobs, logs, and downloadable artifact. This evidence complements the committed workflow definition rather than treating configuration alone as proof of execution.

Several limitations govern interpretation. Exact direct-edge matching cannot credit a synonymous label or an equivalent ancestor relation. An incomplete reference can make a valid generated relation appear false. Scores are ranking values, not necessarily calibrated probabilities. CSV cannot preserve concepts that never appear in any relation, so complete isolated-node analysis requires JSON with all nodes exported.

The demonstration supports implementation correctness on documented examples, not a research conclusion about taxonomy induction. No real dissertation corpus, expert annotation study, uncertainty interval, or method comparison is claimed. For an actual experiment, report the annotation protocol, frozen threshold, input availability, software commit, and qualitative error analysis with the numerical metrics.

10. Conclusion, self-analysis, and references

The completed work turns a dissertation-related idea into a separate, bounded scientific project. Its contribution is an independent export-based comparison with a reference taxonomy, combined with graph checks and reproducible reports. Git history, public documentation, tests, and automated artifact delivery make the procedure inspectable and runnable by another researcher.

The main design challenge was avoiding duplication of evaluation already present in Taxonomy Builder. Source inspection identified external gold-standard comparison as a useful separate responsibility. Other challenges were ID-based exports, ambiguous normalized labels, undefined metrics, and longest-path depth in graphs with multiple parents. Explicit conventions and focused regression checks address these cases. The remaining scientific challenge is obtaining a defensible gold standard; software alone cannot solve it.

References and implementation evidence

- [1] thgank. *Taxonomy Builder*. Inspected source revision adc387a5. Sources include the API export service, export schema, worker evaluation, and QA workflow.
- [2] Chacon, S., and Straub, B. *Pro Git*, section 1.1: About Version Control. Official Git book.
- [3] GitHub. *Understanding GitHub Actions*. Official workflow documentation.
- [4] Python Software Foundation. *graphlib: Functionality to operate with graph-like structures*. Python documentation.
- [5] Python Software Foundation. *unittest: Unit testing framework*. Python documentation.
- [6] Python Software Foundation. *unicodedata: Unicode Database*. Normalization documentation.
- [7] Wilson, G., et al. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. doi:10.1371/journal.pcbi.1005510.
- [8] Apache Software Foundation. *Apache Subversion Features*. Official project documentation.
- [9] Mercurial. *About Mercurial*. Official project overview.
- [10] GitHub. *Secure use reference*. Workflow security documentation.

Sources accessed 6 October 2026. Practical submission: Taxonomy Evaluator; CI/CD configuration.